```
    using (var stringReader = new StreamReader(bodyStream))        ❺
    {                                                              ❺
        response = await stringReader.ReadToEndAsync();            ❺
    }                                                              ❺

    Assert.Equal("pong", response);                                ❻
    Assert.Equal("text/plain", context.Response.ContentType);      ❼
    Assert.Equal(200, context.Response.StatusCode);                ❽
}
```

❶ Creates a DefaultHttpContext and initializes the body with a MemoryStream

❷ The path is set to the required value for the StatusMiddleware.

❸ Creates an instance of the middleware and passes in a simple RequestDelegate

❹ Invokes the middleware

❺ Rewinds the MemoryStream and reads the response body into a string

❻ Verifies that the response has the correct value

❼ Verifies that the ContentType response is correct

❽ Verifies that the Status Code response is correct

As you can see, unit testing middleware requires a lot of setup. On the positive side, it allows you to test your middleware in isolation, but in some cases, especially for simple middleware without any dependencies on databases or other services, integration testing can (somewhat surprisingly) be easier. In section 36.3 you'll create integration tests for this middleware to see the difference.

Custom middleware is common in ASP.NET Core projects, but far more common are Razor Pages, API controllers, and minimal API endpoints. In the next section you'll see how you can unit test them in isolation from other components.

36.2 Unit testing API controllers and minimal API endpoints